INTRODUÇÃO A PWN
(NÍVEL BÁSICO)

1 - Introdução

1.1 – Do que se trata?

O termo PWN, derivado da palavra own (possuir/dominar) refere-se a uma disciplina avançada da segurança ofensiva focada na identificação e exploração de vulnerabilidades em programas compilados (binários). Diferente de outras vertentes da cybersegurança, o PWN atua no nível da arquitetura de sistemas, manipulando a forma como o software interage diretamente com a CPU e a Memória RAM.

O objetivo fundamental de uma exploração de PWN é subverter o fluxo de controle original de um processo. Ao identificar falhas na implementação lógica ou no gerenciamento de memória de um executável, é possível forçar o sistema a executar instruções arbitrárias, resultando, na maioria dos casos, na obtenção de um *shell* (terminal de comando) com privilégios de acesso que eram pra ser bloqueados.

1.2 – Fundamentos técnicos

A prática de PWN exige a compreensão profunda de como o sistema operacional organiza a execução de um programa. A memória de um processo é estruturada em segmentos distintos, sendo os mais críticos para o estudo de vulnerabilidades:

- **Stack (Pilha):** Área de memória de curto prazo que armazena variáveis locais, argumentos de funções e, crucialmente, os endereços de retorno que indicam para onde o programa deve voltar após finalizar uma função.
- **Heap (Monte):** Segmento destinado à alocação dinâmica de memória, utilizado para dados cujo tamanho ou tempo de vida não podem ser previstos antes da execução.

2 – Modelos de arquitetura

2.1 – Introdução a arquiteturas x86 e x64

A CPU segue a Arquitetura Von Neumann, sendo composta por unidades de controle (que buscam instruções na RAM), unidades de lógica aritmética (ALU, que executam as instruções) e registradores. A CPU interage diretamente com a memória RAM, que armazena todo o código e dados necessários para a execução de um programa, e com dispositivos de entrada/saída.

De uma maneira mais detalhada, os componentes principais de um sistema são:

- **Unidade de Controle**: Responsável por captar as diretrizes armazenadas na memória RAM.
- **Unidade de Lógica e Aritmética**: Encarregada do processamento das instruções coletadas, direcionando os dados resultantes para a memória principal ou para os registradores internos.
- **Registros**: Pequenas unidades de memória interna da CPU que otimizam o tempo de processamento ao manter informações essenciais prontas para uso imediato.
- **Memória RAM**: Espaço de armazenamento volátil que hospeda o conjunto de instruções e as informações necessárias para a execução de qualquer software.

- **Periféricos de Entrada/Saída**: Conjunto de hardwares externos e internos, como periféricos de controle e unidades de armazenamento, que permitem a comunicação com o computador.

A compreensão dessa arquitetura é vital, pois a maioria das vulnerabilidades de PWN surge da manipulação indevida desse fluxo de dados entre a Memória RAM e a CPU. Como a Unidade de Controle busca instruções diretamente da memória principal, um atacante que consiga corromper dados na RAM pode enganar a CPU, fazendo-a processar instruções maliciosas como se fossem legítimas. Esse controle é exercido através da manipulação de registradores críticos, que funcionam como os "ponteiros" que guiam o comportamento do processador durante a execução de um software.

2.2 – Registradores

Os registradores são espaços de armazenamento extremamente rápidos dentro da CPU, usados para economizar tempo no acesso a dados críticos. Eles são divididos em:

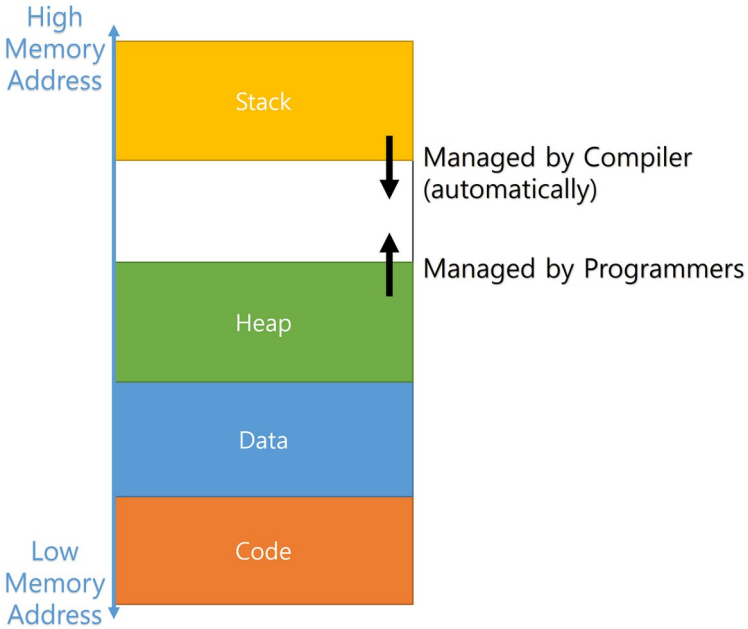
- **Ponteiros de Instrução (EIP/RIP):** Armazenam o endereço da próxima instrução a ser executada.
- **Registradores de Uso Geral:** São usados durante a execução geral das instruções da CPU. Contém os seguintes registradores:
  - **EAX/RAX** (acumulador): resultados das operações aritméticas são frequentemente armazenados neste registro.
  - **EBX/RBX** (registro base): usado para armazenar o endereço base para referenciar um deslocamento.
  - **ECX/RCX** (registro contador): usado em operações de contagem, como loops, entre outros.
  - **EDX/RDX** (registro de dados): usado em operações de multiplicação e divisão.
  - **ESP/RSP** (ponteiro da stack): aponta para o topo da pilha Stack.
  - **EBP/RBP** (ponteiro de base): é usado para acessar parâmetros passados pela pilha.
  - **ESI/RSI** (registro source index): usado para operações de string.
  - **EDI/RDI** (registro destination index): usado também para operações de string.
  - **R8-R15**: existem apenas em sistemas x64.
- **Status Flags (EFLAGS/RFLAGS):** Bits que indicam o status de uma operação, como o Zero Flag (resultado zero), Carry Flag (número muito grande/pequeno), Sign Flag (resultado negativo) e Trap Flag (indica se o processador está em modo debug).
- **Registradores de Segmento:** Registradores de segmento são registradores de 16 bits que convertem o espaço de memória plana em diferentes segmentos para facilitar o endereçamento. São eles:
  - Code Segment: aponta para a seção de código na memória.

- Data Segment: aponta para a seção de dados do programa na memória.
- Stack Segment: aponta para a Stack do programa na memória.
- Extra Segment: apontam para diferentes seções de dados.

2.3 – Estrutura da memória:

Ao ser executado, um programa não interage diretamente com os endereços físicos da memória RAM. Em vez disso, o Sistema Operacional fornece uma **abstração** conhecida como **Memória Virtual**. Este mecanismo cria a ilusão de que o processo possui um espaço de endereçamento contínuo, privado e vasto, simplificando a execução e garantindo a segurança do sistema.

Aqui está um exemplo da estrutura da memória:



Nessa estrutura, é possível analisar alguns componentes. São eles:

Code (Código)

Localizada nos endereços mais baixos da memória, esta seção armazena as instruções binárias que a CPU deve executar.

- **Função**: Contém o texto do programa compilado (instruções executáveis).
- **Permissões**: Possui permissões de execução, permitindo que o processador processe os dados ali contidos como ordens de comando.

2. Data (Dados)

Esta região armazena informações que permanecem constantes ou que têm um tempo de vida global dentro do programa.

- **Função**: Contém variáveis globais e dados inicializados que não devem sofrer alterações estruturais durante a execução.

3. Heap (Monte)

O **Heap** é a área de memória dinâmica, cujo tamanho cresce em direção aos endereços mais altos (conforme indicado pela seta na imagem).

- **Gerenciamento**: Como indica a imagem acima, esta seção é gerenciada por programadores.
- **Função**: Variáveis e estruturas de dados são criadas aqui em tempo de execução e devem ser explicitamente liberadas após o uso para evitar vazamentos de memória.

4. Stack (Pilha)

A Stack é uma estrutura de memória LIFO (Last-In, First-Out) que cresce em direção aos endereços mais baixos, partindo do topo da memória.

- **Gerenciamento**: É gerenciada automaticamente pelo compilador.
- **Função**: Armazena variáveis locais de funções, argumentos passados para o programa e, crucialmente, o endereço de retorno (Return Address). Porém, como o endereço de retorno é diretamente relacionado com o fluxo de instruções da CPU, a stack é muito utilizada por malwares para se "sequestrar" o fluxo de controle.

2.4 – Estrutura da pilha (Stack):

A Stack é um segmento de memória fundamental que organiza os dados necessários para a execução de funções, incluindo argumentos, variáveis locais e as diretrizes do fluxo de controle. Ela opera sob a lógica LIFO (*Last-In, First-Out*), garantindo que o dado inserido mais recentemente seja o primeiro a ser processado.

Para gerenciar essa estrutura, a CPU utiliza dois registradores principais:

- **EBP/RBP (Base Pointer):** Atua como um ponto de referência fixo e constante para o quadro de pilha atual, permitindo que o programa localize suas variáveis e parâmetros de forma estável.
- **ESP/RSP (Stack Pointer):** Responsável por apontar dinamicamente para o topo atual da pilha.

No layout da pilha, a organização abaixo do ponteiro de base (BP) é crítica para a segurança pois:

1. Imediatamente abaixo do BP atual, armazena-se o BP do program1a anterior (chamador) para restaurar o contexto após a execução.
2. Logo abaixo deste, encontra-se o Endereço de Retorno (Return Address), que instrui o Ponteiro de Instrução (IP) sobre onde retomar a execução assim que a função atual for finalizada.

A segurança de um binário reside na integridade da comunicação entre a CPU e a Memória RAM. Conforme analisado, a Stack é o componente mais sensível deste ecossistema, pois além de gerenciar dados temporários e variáveis locais, ela armazena o endereço de retorno que dita o fluxo de execução do programa.

A manipulação precisa dos registradores de controle (RSP/RBP) e a lógica LIFO garantem a previsibilidade do software. Esse é o ponto principal onde são encontradas as vulnerabilidades que colocam em risco a integridade do sistema.

2.4 – Programação para sistemas x86 (Assembly)

A linguagem Assembly representa o nível mais baixo de abstração legível por seres humanos, funcionando como uma ponte direta para os códigos de operação (*opcodes*) executados pelo hardware. No contexto de segurança ofensiva, o domínio desta linguagem permite ao analista compreender a manipulação precisa de dados na memória e identificar desvios lógicos passíveis de exploração.

I. Estrutura das Instruções e Operandos:

Cada instrução Assembly diz à CPU qual operação realizar, utilizando diferentes tipos de operandos para processar e armazenar resultados:

- **Imediatos:** Representam valores constantes fixos e injetados diretamente na instrução (ex: mov, eax, 10).
- **Registradores**: São os operandos mais rápidos, permitindo que a CPU processe dados internamente sem acessar o barramento de memória externa.
- **Memória:** Identificados por colchetes `[]`, estes operandos realizam referências diretas a endereços de memória, permitindo ler ou escrever dados na RAM.

II. Instruções de Manipulação de dados:

Estas instruções são responsáveis pelo fluxo de dados entre os componentes da arquitetura Von Neumann, movendo informações entre registradores e memória.

- **MOV**: Esta instrução realiza a cópia de um valor de uma origem para um destino, podendo transitar dados entre dois registradores, entre memória e registrador, ou carregar um valor imediato.
- **LEA (Load Effective Address)**: Diferente do MOV, o LEA calcula e carrega o **endereço** resultante de uma expressão de memória para um registrador de destino. É frequentemente utilizada por compiladores para realizar operações aritméticas complexas em uma única instrução sem alterar as *flags* de status.
- **NOP (No Operation)**: Instrução que não altera o estado dos dados, apenas consome ciclos de CPU. Na exploração de binários, é utilizada para criar o *NOP Sled*, uma sequência que garante que a execução atinja o *shellcode* de forma estável, mesmo que o endereço de salto não seja exato.

III. Instruções Aritméticas e Lógicas:

A Unidade de Lógica e Aritmética (ALU) processa estes comandos para realizar transformações de dados e cálculos de deslocamento.

- **Aritmética**: Inclui operações básicas como soma (ADD) e subtração (SUB), além de instruções específicas de multiplicação (MUL) e divisão (DIV) que utilizam os registradores EAX e EDX para acomodar resultados de 64 bits.
- **Incremento/Decremento**: As instruções INC e DEC alteram o valor de um registrador em exatamente 1 unidade, sendo fundamentais para o controle de contadores em loops.
- **Operações Lógicas**: Instruções como AND, OR e NOT realizam manipulações bit a bit. A instrução XOR é amplamente usada para zerar um registrador (ex: xor eax, eax), pois é mais eficiente em termos de processamento e tamanho de byte do que um *mov eax, 0*.

IV. Controle de Fluxo e Ramificações:

O controle de fluxo permite que o programa tome decisões e saia da execução linear ao alterar o valor do Ponteiro de Instrução (EIP/RIP).

- **JMP (Salto Incondicional)**: Força a CPU a carregar um novo endereço no ponteiro de instrução, mudando o local da próxima busca de comando na RAM.
- **Saltos Condicionais**: Dependem do estado dos *Status Flags* (como Zero Flag ou Carry Flag) definidos por instruções anteriores como `CMP` ou `TEST`. Exemplos incluem `JE` (salta se igual) e `JNZ` (salta se o resultado não for zero).

Instruction	Description	signed-ness	Flags	short jump opcodes	near jump opcodes
JO	Jump if overflow		OF = 1	70	0F 80
JNO	Jump if not overflow		OF = 0	71	0F 81
JS	Jump if sign		SF = 1	78	0F 88
JNS	Jump if not sign		SF = 0	79	0F 89
JE JZ	Jump if equal Jump if zero		ZF = 1	74	0F 84
JNE JNZ	Jump if not equal Jump if not zero		ZF = 0	75	0F 85
JB JNAE JC	Jump if below Jump if not above or equal Jump if carry	unsigned	CF = 1	72	0F 82
JNB JAE JNC	Jump if not below Jump if above or equal Jump if not carry	unsigned	CF = 0	73	0F 83
JBE JNA	Jump if below or equal Jump if not above	unsigned	CF = 1 or ZF = 1	76	0F 86
JA JNBE	Jump if above Jump if not below or equal	unsigned	CF = 0 and ZF = 0	77	0F 87
JL JNGE	Jump if less Jump if not greater or equal	signed	SF <> OF	7C	0F 8C
JGE JNL	Jump if greater or equal Jump if not less	signed	SF = OF	7D	0F 8D
JLE JNG	Jump if less or equal Jump if not greater	signed	ZF = 1 or SF <> OF	7E	0F 8E
JG JNLE	Jump if greater Jump if not less or equal	signed	ZF = 0 and SF = OF	7F	0F 8F
JP JPE	Jump if parity Jump if parity even		PF = 1	7A	0F 8A
JNP JPO	Jump if not parity Jump if parity odd		PF = 0	7B	0F 8B
JCXZ JECXZ	Jump if %CX register is 0 Jump if %ECX register is 0		%CX = 0 %ECX = 0	E3	

- **CALL**: Inicia a execução de uma função, desviando o fluxo para um novo endereço, mas garantindo que o endereço de retorno seja salvo na Stack para que o programa possa retomar sua posição original após a conclusão.

V. Manipulação direto da Stack:

A interação com a Stack é feita através de operações que modificam automaticamente o Stack Pointer (ESP/RSP).

- **PUSH**: Armazena o operando no topo da pilha, decrementando o ponteiro de pilha para abrir espaço na memória. Existem variantes como `PUSHA` que salvam todos os registradores de uso geral simultaneamente.
- **POP**: Recupera o dado do topo da Stack, armazena-o no registrador de destino e incrementa o ponteiro de pilha para refletir a retirada do valor.

VI. Flags de estado:

As flags de estado residem no registrador **EFLAGS** (x86) ou **RFLAGS** (x64). Elas funcionam como “interruptores” de 1 bit que a CPU utiliza para sinalizar o resultado da operação aritmética ou lógica mais recente.

- **Zero Flag (ZF):** É ativada (1) sempre que o resultado de uma operação for exatamente zero. É a flag mais utilizada em saltos condicionais após instruções de comparação (`CMP`) ou teste (`TEST`).
- **Carry Flag (CF):** Sinaliza quando uma operação aritmética gera um "vai-um" (transbordo) para fora do bit mais significativo, indicando que o resultado é muito grande para o destino.
- **Sign Flag (SF):** Reflete o estado do bit mais significativo do resultado; se for 1, o resultado é tratado como negativo.
- **Overflow Flag (OF):** Diferente da CF, a OF indica um erro de transbordo em operações com sinal (ex: somar dois números positivos e obter um resultado negativo).

Flags de estado

Nome	Símbolo	Função/característica
Carry Flag	CF	Indicador de "vai-um"
Parity Flag	PF	Indicador de número PAR de 1's no byte inferior
Auxiliary Carry	AF	Indicador de "vai-um" para operações em BCD
Zero Flag	ZF	Indicador de "zero" na última operação
Sign Flag	SF	Indicador de resultado negativo
Overflow Flag	OF	Indicador de erro de transbordamento

VII. Condicionais:

As condicionais fornecem a lógica necessária para que a CPU tome decisões e desvie de uma execução linear com base em estados e resultados de operações anteriores.

- **TEST:** Realiza uma operação lógica AND bit a bit entre dois operandos para verificar condições específicas sem alterar o valor do destino. É amplamente utilizada para verificar se um registrador contém um valor nulo (NULL), pois define a *Zero Flag* ($ZF=1$) se o resultado for zero, sendo mais eficiente em termos de bytes do que uma comparação direta.
- **CMP (Compare):** Compara dois operandos executando uma subtração interna que não armazena o resultado, servindo exclusivamente para configurar as flags de estado. Se os valores forem idênticos, a *Zero Flag* (ZF) é ativada; se o operando de origem for maior que o de destino, a *Carry Flag* (CF) é definida.

- **Saltos Condicionais (Jcc):** São instruções de ramificação que alteram o valor do Ponteiro de Instrução (IP) apenas se uma condição de flag for atendida. (Tabela encontrada no tópico IV.)

3 – Buffer Overflow

3.1 – O que é Buffer Overflow?

É uma vulnerabilidade que ocorre quando um programa escreve dados em um armazenamento temporário (*buffer*) além da sua capacidade de alocação. Esse excedente transborda para os espaços de memória adjacentes, podendo causar comportamentos imprevistos ou falhas de segurança.

Essa falha acaba por ocasionar um erro conhecido como segmentation fault, permitindo que o atacante possa acessar endereços de memória diferentes, ou enviar comandos através de código.

3.2 – O que é Segmentation Fault?

O Segmentation Fault, ou *segfault*, é um erro crítico gerado pelo hardware (MMU) e notificado pelo Sistema Operacional quando um processo tenta acessar uma “página” ou segmento de memória para o qual não possui permissão ou que não está mapeado em seu espaço de endereçamento virtual.

Diferente de erros lógicos simples, o *segfault* ocorre em nível de proteção de memória. Os sistemas modernos utilizam sinalizadores específicos para cada segmento, garantindo a integridade do sistema:

- **Permissões de Acesso:** Cada região da memória (Stack, Heap, Code) possui atributos de Leitura (R), Escrita (W) e Execução (X). Se um programa tenta, por exemplo, escrever em uma região marcada apenas como leitura (como o segmento de *Código*), o kernel interrompe a execução imediatamente para evitar a corrupção de instruções.
- **Ponteiros Inválidos e Nulos:** Uma causa comum de *segfault* é a desreferenciação de ponteiros nulos ou “soltos” (*dangling pointers*), que apontam para endereços que o processo não possui ou que foram devolvidos ao sistema.
- **O papel no Buffer Overflow:** No contexto de exploração, o *segfault* é frequentemente o primeiro sinal de uma vulnerabilidade. Quando transbordamos um buffer na Stack, acabamos sobrescrevendo o *Endereço de Retorno* com valores arbitrários (como uma sequência de 'A's). Quando a função termina e tenta pular para esse endereço inexistente ou protegido (ex: `0x41414141`), a CPU detecta a violação de acesso e encerra o programa com uma falha de segmentação.

3.2 – Tipos de vulnerabilidades em C

O foco da vulnerabilidade de *Buffer Overflow* reside no uso de funções que priorizam o desempenho em detrimento da segurança, falhando em realizar a verificação de limites (*bounds checking*). Em C, muitas funções de manipulação de strings e entrada de dados assumem que o programador forneceu um buffer de destino grande o suficiente para acomodar qualquer entrada.

Aqui estão alguns exemplos de onde ocorre o buffer overflow

I. Entrada de Dados Direta

Algumas funções de leitura são intrinsecamente inseguras por design, pois não possuem argumentos para limitar a quantidade de bytes lidos:

- **gets()**: Considerada a função mais perigosa da biblioteca padrão. Ela lê a entrada do usuário até encontrar um caractere de nova linha (\n) ou o fim do arquivo (EOF), ignorando completamente o tamanho do buffer de destino.
- **scanf() com `%s`**: Quando usada sem um especificador de largura (ex: `%10s`), a função `scanf` continua lendo caracteres indefinidamente até encontrar um espaço em branco, tornando-se um vetor comum de transbordamento.

II. Manipulação de Strings sem Verificação de Limites:

Funções que processam ou copiam strings na memória costumam confiar no caractere nulo (`\0`) como terminador, o que pode ser explorado para sobrescrever dados adjacentes:

- **strcpy(dest, src)**: Copia a string de origem para o destino até encontrar o byte nulo. Se a origem for maior que o espaço alocado no destino, os bytes excedentes sobrescreverão o restante da Stack, incluindo o endereço de retorno.
- **strcat()**: Concatena uma string ao final de outra sem verificar se o buffer resultante excede o limite máximo alocado para a variável de destino.
- **sprintf()**: Utilizada para formatar e armazenar strings em um buffer. Sem o controle rigoroso da largura dos argumentos formatados, é fácil transbordar o buffer de saída.

3.3 – Tipos de Buffer Overflow

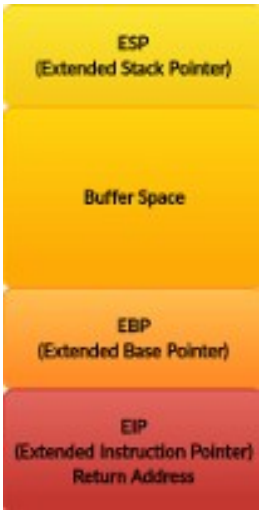
As vulnerabilidades de transbordamento são classificadas de acordo com a região da memória afetada e a técnica de manipulação utilizada. Enquanto o foco na Stack é a corrupção de endereços de retorno, as falhas em Heap exploram a memória dinâmica para subverter a lógica do software.

Aqui estão explicadas de uma maneira mais detalhada cada um dos dois tipos:

I. Stack:

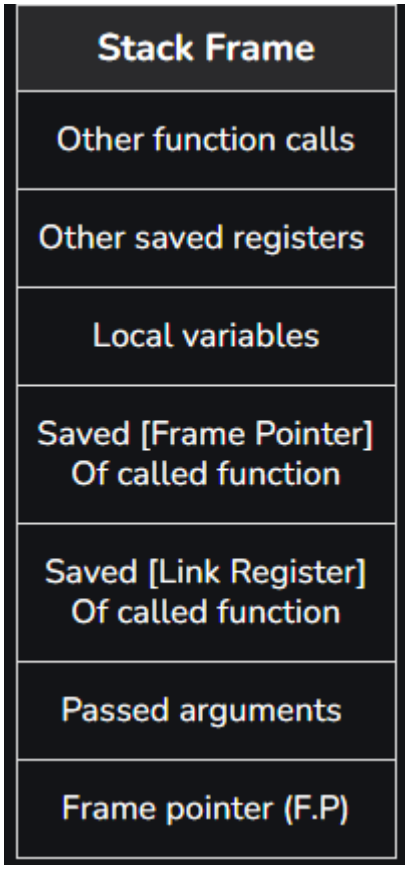
A *Stack* é uma região da memória RAM organizada sob a lógica *LIFO* (*Last-In, First-Out*), onde os dados são empilhados e removidos automaticamente pelo sistema durante a execução de funções. Ela armazena variáveis locais, parâmetros e o endereço de retorno, operando dentro de um limite fixo de tamanho.

O controle dessa estrutura é feito por registradores específicos: o *EBP* (ou RBP em 64 bits) aponta para a base fixa da pilha atual, enquanto o *ESP* (ou RSP) aponta para o topo dinâmico. Complementando o fluxo, o *EIP* (ou RIP) indica a próxima instrução a ser executada, e o *EAX* (ou RAX) frequentemente armazena o endereço de retorno da função. O *Buffer Overflow* ocorre justamente quando a escrita excede esses limites da stack, permitindo sobrescrever esses registradores e desviar o curso original do programa.



Register	16 bits	32 bits	64 bits	Type
Accumulator	AX	EAX	RAX	General
Base	BX	EBX	RBX	General
Counter	CX	ECX	RCX	General
Data	DX	EDX	RDX	General
Source Index	SI	ESI	RSI	Pointer
Target Index	DI	EDI	RDI	Pointer
Base Pointer	BP	EBP	RBP	Pointer
Top Pointer	SP	ESP	RSP	Pointer
Instruction Pointer	IP	EIP	RIP	Pointer
Code Segment	CS	-	-	Segmen

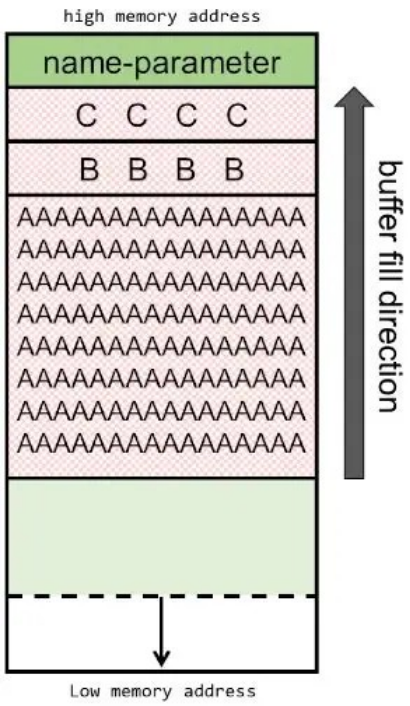
- ✗ A cada chamada de uma função, o sistema operacional reserva um bloco específico na memória RAM denominado Stack Frame. Este segmento é fundamental para o isolamento da execução, pois consolida as informações sobre os parâmetros recebidos, as variáveis locais declaradas e o endereço de retorno para a função de origem.
- ✗ A alocação dentro do Stack Frame segue uma ordem rigorosa e sequencial: inicialmente, são empilhados os parâmetros passados para a função; em seguida, grava-se o endereço de retorno (Return Address), que instrui o processador sobre onde retomar a execução após o encerramento do método; por fim, são alocadas as variáveis locais utilizadas pela função. Este processo garante que, ao finalizar a tarefa, o programa recupere o contexto anterior e prossiga com o fluxo de controle original.



Detalhamento do Stack Frame ao lado:

- **Other function calls:** Refere-se ao espaço reservado ou empilhamento de dados necessários para chamadas de novas funções que ocorram dentro do método atual.
- **Other saved registers:** Área destinada a preservar o estado de registradores que a função atual precisa utilizar, garantindo que seus valores originais sejam restaurados ao final da execução.
- **Local variables:** Segmento onde são armazenadas as variáveis declaradas dentro do escopo da própria função. É aqui que geralmente residem os buffers (armazenamentos temporários) que podem ser alvos de transbordamento.
- **Saved [Frame Pointer] of called function:** Armazena o endereço do ponteiro de base (*EBP/RBP*) da função que realizou a chamada (função pai). Isso permite que, ao terminar, a CPU saiba exatamente onde começava o quadro de pilha anterior.
- **Saved [Link Register] of called function:** No contexto de exploração e arquiteturas específicas, refere-se ao *Return Address* (Endereço de Retorno). É a informação mais crítica: o endereço da próxima instrução que a CPU deve executar após a função atual ser finalizada.
- **Passed arguments:** Contém os parâmetros ou argumentos que foram passados para a função no momento de sua chamada. Eles são as primeiras informações alocadas ao criar o frame.
- **Frame pointer (F.P):** Representa o registrador de base atual (**EBP/RBP**). Ele serve como o ponto de referência estático a partir do qual o programa localiza seus argumentos e variáveis locais através de deslocamentos (offsets).

Exemplo de Stack Overflow:



Detalhamento da stack:

- **High e Low Memory Address:** A memória é representada com endereços altos no topo e baixos na base. Note que a seta central aponta para baixo, indicando que a Stack cresce em direção aos endereços baixos.
- **Buffer Fill Direction:** A seta lateral à direita mostra que, embora a Stack cresça para baixo, o preenchimento de um buffer (como uma string) ocorre de forma sequencial, dos endereços menores para os maiores.
- **name-parameter (Verde):** No topo da pilha (endereço mais alto), estão os argumentos ou parâmetros passados para a função.
- **C C C C (Rosa claro):** Representa o Return Address (Endereço de Retorno) salvo. Este é o valor que diz à CPU para onde voltar após a função terminar.
- **B B B B (Rosa claro):** Representa o Saved EBP (Base Pointer), o registro que mantém a referência da base da pilha anterior.
- **Bloco de A's (Rosa tracejado):** É o espaço reservado originalmente para as variáveis locais (o buffer)

Dinâmica do ataque:

1. O atacante insere dados além da capacidade permitida para as variáveis locais.
2. Como a escrita segue a direção da seta (**buffer fill direction**), os dados começam a invadir o espaço do **Saved EBP** (B B B B).
3. Se o transbordamento continuar, ele atinge e sobrescreve o **Return Address** (C C C C).

Ao sobrescrever o espaço C C C C com um endereço controlado pelo atacante, quando a função chegar à instrução de retorno, a CPU não voltará para o programa original, mas sim saltará para o código malicioso (como um *shellcode*) injetado no buffer.

II. Heap:

Diferente da Stack, que é rígida e automática, a Heap funciona como um grande reservatório de memória compartilhada, oferecendo flexibilidade para o armazenamento de dados cujo tamanho ou tempo de vida não podem ser previstos antes da execução do programa. Enquanto a Stack segue uma ordem estrita, na Heap os dados podem ser alocados e liberados em qualquer ordem, permitindo que informações sobrevivam mesmo após o encerramento da função que as criou.

Para entender o Heap Overflow, imagine que a Heap é um grande depósito compartilhado onde você pode guardar caixas (dados) de qualquer tamanho e por quanto tempo quiser. Diferente da Stack, que é organizada como uma pilha de pratos onde você só mexe no topo, a Heap é um espaço aberto e “livre”.

Aqui está como o overflow funciona de uma maneira mais detalhada:

I. Gerenciamento Manual

Na Heap, o programador (ou o sistema) reserva um espaço, usa e depois precisa avisar que não precisa mais dele.

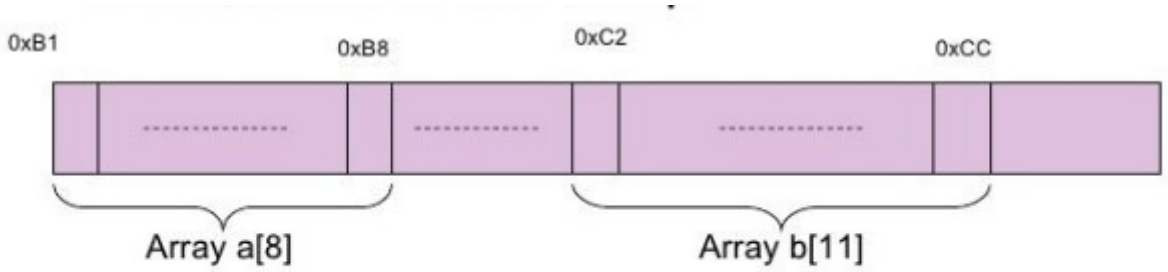
- **Alocação Dinâmica:** Se você precisa criar um array ou objeto cujo tamanho só será descoberto enquanto o programa roda, você usa a Heap.
- **Persistência:** Os dados aqui sobrevivem mesmo depois que a função que os criou termina, o que é ótimo para bancos de dados ou listas de usuários que o programa todo precisa acessar.

II. Como ocorre o Overflow?

O erro ocorre porque, embora a Heap seja flexível, ela ainda divide o espaço em "blocos" vizinhos.

- **O Problema:** Se o programa reserva 100 bytes para o Objeto A e 100 bytes para o Objeto B logo em seguida, e você consegue forçar o programa a escrever 150 bytes no Objeto A, esses 50 bytes extras vão "vazar" e sobrescrever o início do Objeto B.

Exemplo:



- Overflow
  - strcpy(a,long_string);

- **A Função strcpy(a, long_string)**: O exemplo mostra o uso da função `strcpy`, que é considerada perigosa porque não verifica se o texto de origem cabe no destino.
- **Vulnerabilidade**: Se no lugar do Array b estivesse um dado crítico, como um endereço de retorno (Return Address), um atacante poderia usar esse transbordamento para assumir o controle total do programa.

3.4 (Extra) – Exemplos de código com Buffer Overflow:

I. Stack Overflow:

```c
#include <stdio.h>
#include <string.h>

void vulneravel() {
    char buffer[16]; // Espaço reservado na Stack para 16 bytes
    char *segredo = "ENTRADA_MUITO_LONGA_QUE_VAI_ESTOURAR_O_BUFFER";

    // strcpy não verifica limites; vai escrever além dos 16 bytes
    strcpy(buffer, segredo);

    printf("Buffer: %s\n", buffer);
}

int main() {
    vulneravel();
    return 0;
}
```

Explicação Técnica:

- **Alocação**: Ao chamar vulneravel(), um *Stack Frame* é criado para armazenar o buffer[16].
- **O Erro**: A função strcpy copia a string longa para o buffer. Como a string é maior que 16 bytes, ela "sobe" na memória.
- **Consequência**: O excesso de dados sobrescreve o *Saved EBP* e o *Return Address* (Endereço de Retorno) que estão logo acima no frame.
- **Resultado**: Ao tentar retornar da função, a CPU lê um endereço de retorno corrompido, resultando em um *Segmentation Fault* ou permitindo que um atacante redirecione a execução.

II. Heap Overflow:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    // Aloca dois blocos de 16 bytes na Heap
    char *bloco_A = (char *)malloc(16);
    char *bloco_B = (char *)malloc(16);

    strcpy(bloco_B, "DADO_SEGURO");

    // Escrita insegura no bloco_A ultrapassa 16 bytes
    // Isso vai invadir o espaço do bloco_B que é vizinho na memória
    strcpy(bloco_A, "ESTOUREI_O_BLOCO_A_E_VOU_INVADIR_O_B");

    printf("Bloco A: %s\n", bloco_A);
    printf("Bloco B corrompido: %s\n", bloco_B);

    free(bloco_A);
    free(bloco_B);
    return 0;
}
```

Explicação técnica:

- **Alocação**: O malloc reserva espaços na Heap, que é uma área de memória livre e persistente.
- **O Erro**: Como a Heap aloca blocos sequencialmente para otimizar espaço, o bloco_B costuma estar posicionado logo após o bloco_A.
- **Consequência**: Ao escrever dados demais no bloco_A, invadimos o espaço adjacente pertencente ao bloco_B.
- **Diferença Crucial**: No Heap Overflow, não sobrescrevemos um endereço de retorno imediatamente (pois não há um no heap), mas corrompemos dados da aplicação, ponteiros de objetos ou metadados do próprio gerenciador de memória.

4 – Use-After-Free (UAF)

Enquanto os buffer overflows focam na corrupção por excesso de dados, a vulnerabilidade *Use-After-Free* explora falhas na lógica de gerenciamento do ciclo de vida da memória dinâmica (Heap). Esta falha ocorre quando um programa mantém uma referência — o chamado *dangling pointer* (ponteiro solto) — para um endereço de memória que já foi explicitamente liberado pelo sistema.

Diferente da *Stack*, onde o gerenciamento é automático e previsível , a *Heap* exige um controle manual rigoroso por parte do desenvolvedor em linguagens como C e C++. Se um ponteiro continua sendo utilizado após a execução de uma instrução de liberação (como *free()* ou *delete*), abre-se uma brecha crítica para a segurança.

O perigo real do UAF reside na forma como os alocadores de memória modernos operam: para otimizar o desempenho, o sistema frequentemente reutiliza endereços de memória recém-liberados para novas solicitações de dados. Se um atacante conseguir posicionar dados controlados ou objetos maliciosos nesse endereço reaproveitado, o ponteiro original do programa passará a interagir com o conteúdo injetado, permitindo desde o vazamento de informações sensíveis até o sequestro total do fluxo de execução do binário.

4.1 – Ciclo de ataque UAF:

O sucesso de um ataque UAF depende da compreensão de como o gerenciador de memória do sistema operacional (como o *ptmalloc* no Linux) lida com espaços recém-liberados. O processo geralmente segue três fases distintas:

1. **Alocação e Armazenamento (The Setup):** O programa solicita memória dinâmica para um objeto ou estrutura de dados através de *malloc()*. O sistema reserva um bloco na *Heap* e retorna um endereço, que é armazenado em um ponteiro.
   - ➔ Neste estágio, o ponteiro é considerado “vão” e seguro.
2. **Liberação Indevida (The Free):** O programa libera a memória usando free(), mas falha em anular o ponteiro (não define *ptr = NULL*).
   - ➔ **O problema técnico:** Embora o sistema marque aquele bloco de memória como “disponível”, o ponteiro ainda contém o endereço físico original. Temos agora um *Dangling Pointer* (ponteiro solto).
3. **Reutilização e Conflito (The Use):** Como o gerenciador de memória busca eficiência, ele tende a entregar o endereço recém-liberado para a próxima requisição de memória de tamanho semelhante.
   - ➔ Se um atacante conseguir forçar o programa a alocar novos dados (como uma string controlada) logo após o free, o sistema poderá colocar esses dados exatamente onde o “ponteiro solto” ainda está apontando.
   - ➔ Quando o programa original tenta usar aquele ponteiro antigo para realizar uma operação, ele acaba processando os dados injetados pelo atacante em vez do objeto original.
4. **Por que isso é interessante em casos de PWN?**
   - ➔ Se o objeto original continha um *ponteiro de função* (como um método de uma classe), e o atacante sobrescreve esse espaço com um novo endereço, o programa executará o código do atacante assim que tentar chamar a função através do ponteiro antigo.

4.2 – Exemplo Prático: Código Seguro vs. Vulnerável

Para entender como funciona essa falha, precisamos observar o comportamento do ponteiro após a memória ser devolvida ao sistema operacional.

A. Cenário A: Código Vulnerável (UAF):

Neste exemplo, o programa libera a memória, mas mantém o endereço no ponteiro, permitindo que o dado seja acessado indevidamente mais tarde.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char *usuario = (char *)malloc(16); // Alocação na Heap [cite: 57, 369]
    strcpy(usuario, "Admin");

    printf("Usuário atual: %s\n", usuario);

    free(usuario); // Memória liberada [cite: 58, 371]
    // O ponteiro 'usuario' ainda guarda o endereço de memória (Dangling Pointer)

    // ... Outras partes do programa rodam aqui ...

    // USO APÓS LIBERAÇÃO:
    // Se o sistema reutilizou esse espaço, o print abaixo pode exibir dados mali
    printf("Uso indevido do ponteiro liberado: %s\n", usuario);

    return 0;
}
```

B. Cenário B: Código Seguro (Mitigação):

A forma correta de evitar o UAF é anular o ponteiro imediatamente após a liberação. Isso garante que qualquer tentativa de uso posterior cause um erro imediato controlado (Segfault), em vez de permitir uma exploração silenciosa.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char *usuario = (char *)malloc(16); [cite: 57]

    free(usuario); [cite: 58]
    usuario = NULL; // O ponteiro agora aponta para lugar nenhum.

    // Se tentarmos usar 'usuario' aqui, o programa travará de forma segura.
    return 0;
}
```

No PWN, o **Cenário A** é o nosso alvo. Como atacantes, procuramos por momentos em que o desenvolvedor esqueceu de definir o ponteiro como NULL. O objetivo é realizar uma nova alocação (chamada de *Heap Spraying* ou *Heap Grooming*) logo após o free(), para que o sistema coloque o nosso "script malicioso" exatamente onde o ponteiro vulnerável ainda aponta.

5 – GDB e PWNTools

Enquanto o entendimento da Stack e da Heap fornece a base teórica, a prática de PWN exige ferramentas que permitam analisar a memória em tempo real e interagir com binários de forma programática.

Dominar a exploração de binários requer a habilidade de observar a memória enquanto o programa finaliza a execução. O *GDB* é o seu microscópio para análise estática e dinâmica, enquanto o *Pwntools* é a biblioteca que permite automatizar tudo isso.

5.1 – GDB (GNU Debugger)

O GDB é um depurador de baixo nível que permite a execução controlada de um programa. No contexto de PWN, ele é utilizado para monitorar como a Stack e os Registradores se comportam quando submetidos a entradas maliciosas.

A. Conceitos Fundamentais:

Ao depurar um overflow, o usuário deve focar em três áreas principais:

- **Registradores de Fluxo**: Monitorar o EIP/RIP para verificar se ele foi sobrescrito com o endereço desejado.
- **Ponteiros de Stack**: Observar o ESP (topo) e o EBP (base) para entender os limites do *Stack Frame* atual.
- **Disassembly**: Visualizar o código em Assembly para identificar as instruções *call, leave* e *ret*, que são os momentos críticos onde o endereço de retorno é utilizado.

B. Comandos Avançados e Exemplos Práticos:

O GDB contém diversos comandos úteis para se fazer a análise de um binário. Tais comandos podem ser utilizados para uma inspeção aprofundada do fluxo e dos registradores dentro de um binário. A seguir, estão alguns comandos cruciais para a execução:

1. Execução e controle de fluxo:

- ➔ **gdb ./<nome do binário>:** Comando inicial para executar o binário em modo de depuração utilizando o GDB.
- ➔ **disas main:** Desmonta a função main em Assembly. Em contexto de PWN, pode ser utilizado para se obter endereços de funções vulneráveis, como o strcpy, por exemplo.
- ➔ **break *<endereço>**: Define um *breakpoint* em um endereço de memória específico. Isso é crucial quando você não tem o código-fonte e precisa parar a execução logo antes de uma função vulnerável como strcpy ser chamada.
- ➔ **i r eip ebp esp:** Exibe o valor atual dos registradores de controle. Pode ser usado para verificar se o EIP já está apontando para o shellcode a ser executado.
- ➔ **`stepi` (ou `si`)**: Executa exatamente uma instrução de máquina. É usado para avançar passo a passo pelo código assembly, observando como cada registrador muda.

- ➔ `continue` **(ou `c`)**: Retoma a execução do programa até o próximo ponto de parada ou até que ocorra uma falha de segmentação (*segmentation fault*).

2. O Comando 'examine' (x):

Este é o comando mais importante. A sintaxe é *x/nfu <endereço>*, onde `n` é a quantidade, `f` é o formato e `u` é a unidade de medida.

- ➔ **x/20wx $esp**: Exibe as próximas 20 palavras (w) em formato hexadecimal (x) a partir do topo da pilha (ESP). É aqui que você confirma se os "A" (0x41) enviados estão preenchendo a stack corretamente.
- ➔ **x/s $eax**: Interpreta o conteúdo no endereço contido em *EAX* como uma string. Útil para verificar se o dado que você enviou está íntegro na memória.
- ➔ **x/i $eip**: Exibe a instrução de montagem (assembly) que está prestes a ser executada no ponteiro de instrução (EIP)

3. Análise de Registradores e Pilha:

- ➔ **info registers (ou i r)**: Mostra o valor atual de todos os registradores. Em um transbordamento bem-sucedido, você verá o *EBP* ou o *EIP* preenchidos com o seu payload (ex: `0x41414141`).
- ➔ **backtrace full**: Mostra não apenas a hierarquia de funções chamadas, mas também os valores das variáveis locais em cada quadro de pilha (Stack Frame).
- ➔ **p/d $ebp - $esp:** Realiza um cálculo matemático rápido para mostrar a distância em decimal (d) entre a base e o topo da pilha, revelando o tamanho real do frame atual.

4. Manipulação de Dados em Tempo Real:

- ➔ **set {int}0x08049000 = 0xdeadbeef (exemplo):** Permite que você altere manualmente um valor na memória enquanto o programa está pausado. Isso é usado para testar se mudar um endereço de retorno manualmente levaria o programa ao destino desejado.

5. Análise de Falha de Segmentação (Segfault):

Quando o binário trava com um *Segmentation Fault*, o GDB permite identificar a causa exata. Se, ao digitar *info registers*, o valor do *EIP* for 0x41414141, isso prova que você obteve controle total sobre o fluxo de execução, pois o programa tentou executar instruções no endereço correspondente aos seus caracteres '*A*'.

5.2 – Pwntools: Automação e Desenvolvimento de Exploits:

O Pwntools é um framework em Python focado em CTFs e desenvolvimento de exploits para arquiteturas Linux. Ele abstrai tarefas complexas de rede, conversão de formatos de dados e interação com o sistema operacional, permitindo que o foco seja apenas na lógica da vulnerabilidade.

A. Gerenciamento de Processos e Conexões:
A biblioteca permite alternar entre um ambiente local e um servidor remoto com apenas uma mudança de linha:

- ➔ **p = process('./binario'):** Inicia o binário localmente para testes e depuração.

- ➔ **r = remote('IP_ALVO', PORTA)**: Conecta-se à instância do desafio hospedada em um servidor
- ➔ **p.interactive()**: Abre um terminal de interação direta com o programa após o disparo do payload. É essencial para usar o shell obtido.

B. Funções de Empacotamento (Packing):
A memória do computador armazena dados em formatos específicos, como o *Little Endian*. O Pwntools lida com isso automaticamente:

- ➔ **p32(0x080484b6)**: Converte um endereço de 32 bits para o formato binário correto (4 bytes) para sobrescrever registradores como *EAX* ou *EIP*.
- ➔ **p64(0x00007fffffffe120)**: Versão para arquiteturas de 64 bits (8 bytes).
- ➔ **u32() / u64()**: Realiza a operação inversa (*unpack*), transformando bytes recebidos de volta em endereços legíveis.

C. Manipulação de Entradas (Receiving and Sending):

Em vez de lidar com *stdin* e *stdout* manualmente, o Pwntools oferece métodos robustos:

- ➔ **p.recvuntil(b"Digite algo:")**: Pausa o script e aguarda o programa imprimir uma string específica antes de enviar o payload.
- ➔ **p.sendline(payload)**: Envia o payload seguido de uma quebra de linha (`\n`), simulando o uso das funções `gets()` ou `scanf()`.
- ➔ **p.send(payload)**: Envia os dados brutos, sem adicionar caracteres extras.

D. Utilitários de Exploração (ELF e Cyclic):

O Pwntools possui ferramentas internas para análise do binário:

- ➔ **elf = ELF('./binario')**: Carrega as propriedades do arquivo, permitindo encontrar endereços de funções (ex: elf.symbols['system']) automaticamente.
- ➔ **cyclic(500)**: Gera um padrão único de 500 caracteres. Ao enviar isso e ver o programa travar, você pode usar *cyclic_find(0x61616167)* para descobrir a distância (*offset*) exata até o endereço de retorno.

E. Exemplo de Script:

```python
from pwn import *

# Configuração do contexto (arquitetura e logs)
context.arch = 'i386'
context.log_level = 'debug'

# 1. Alvo
p = process('./vulneravel')

# 2. Construção do Payload
offset = 44  # Distância descoberta no GDB
target_addr = 0x080484b6 # Endereço da função de shell ou endereço de retorno

payload = flat(
    b"A" * offset,
    target_addr
)

# 3. Disparo e Interação
p.recvuntil(b"entrada:")
p.sendline(payload)
p.interactive()
```

O script utiliza o Pwntools para automatizar um *Buffer Overflow*, configurando a arquitetura e carregando o binário alvo. Ele cria um *payload* que preenche a pilha com caracteres de "lixo" até sobrescrever o endereço de retorno (EIP) com um destino controlado. Por fim, dispara a carga no momento certo e abre o modo interativo para que o atacante controle o sistema.